14 Testing beyond JUnit

This chapter covers

- Integration testing with Testcontainers
- Specification-style testing with Spek and Kotlin
- Property-based testing with Clojure

In the previous chapter, we looked at the general principles that guide our testing. Now we're going to dive deeper into specific approaches to improve our testing for different situations. Whether our goal is cleaner testing of dependencies, better communication in our testing code, or even discovery of edge cases we hadn't considered, the JVM ecosystem provides many tools to help out, and we will highlight only a few. Let's start with that ever-present struggle: how to deal with external dependencies effectively when writing integration tests.

14.1 Integration testing with Testcontainers

As we move up the pyramid from our isolated unit tests, we encounter a variety of obstacles. To integration test against a real database requires that we have a real database available to use! Getting the benefits of that realistic testing implies a huge increase in setup complexity. The statefulness of these external systems also increases the chances of our tests failing, not because of problems with our code but because of unexpected state lingering between tests.

Over the years, this has been tackled in many ways, from in-memory databases to frameworks for running tests fully within transactions that clean up after themselves. But these solutions often bring their own edge cases and difficulties.

Containerization technology, as discussed in chapter 12, provides an interesting new approach to the problem. Because containers are ephemeral, they are well suited to spinning up for a given test run. Because they encapsulate the real databases and other services we want to interact with, they avoid the subtle mismatches substitutes that in-memory databases are prone to.

4.1.1 Installing testcontainers

One of the simplest ways to leverage containers in our testing is through the `testcontainers` library (see **https://www.testcontainers.org/**). This provides an API to control containers directly from our test code, with a wide variety of supported modules for common dependencies. The core functionality is provided through the `org.testcontainers.testcontainers` JAR in Maven:

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers</artifactId>
  <version>1.15.3</version>
  <scope>test</scope>
</dependency>
```

or Gradle:

```
testImplementation "org.testcontainers:testcontainers:1.15.3"
```

4.1.2 An example with Redis

If you recall, we left our theater application downloading prices from an HTTP service. We'd like to introduce a cache for those values. Although proper caching is a whole topic of its own, imagine we decided to externalize the cache rather than just putting the values in memory. A typical datastore for this is Redis (**https://redis.io/**). Redis exposes blazing fast access to get, set, and delete key-value pairs, along with other more complex data structures.

The `Price` interface that we already introduced for data lookups from an HTTP service, shown next, allows us the flexibility to add the caching as a separate concern:

```
package com.wellgrounded;

import redis.clients.jedis.Jedis;

import java.math.BigDecimal;

public class CachedPrice implements Price {
    private final Price priceLookup;
    private final Jedis cacheClient;
```

```
    private static final String priceKey = "price";      ❶

    CachedPrice(Price priceLookup, Jedis cacheClient) {   ❷
      this.priceLookup = priceLookup;
      this.cacheClient = cacheClient;
    }

    @Override
    public BigDecimal getInitialPrice() {
      String cachedPrice = cacheClient.get(priceKey);   ❸
      if (cachedPrice != null) {
        return new BigDecimal(cachedPrice);
      }

      BigDecimal price =
        priceLookup.getInitialPrice();              ❹
      cacheClient.set(priceKey,
                price.toPlainString());          ❺
      return price;
    }
  }
```

❶ The name of the key we will cache the price in Redis

❷ We use the Jedis (**https://github.com/redis/jedis**) library for access to Redis.

❸ Checks whether the cache has this price already

❹ If we don't have the price, uses the lookup provided

❺ Caches the value we just retrieved

At this point it's worth pausing to consider what aspect of the system we want to test. The main point of the `CachedPrice` class is the interaction between Redis and our underlying price lookup. How we work with Redis is key, and Testcontainers lets us test against the real thing as follows:

```
package com.wellgrounded;

import org.junit.jupiter.api.Test;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.junit.jupiter.*;
import org.testcontainers.utility.DockerImageName;
```

```
import redis.clients.jedis.*;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.assertEquals;

@Testcontainers
public class CachedPriceTest {
    private static final DockerImageName imageName =
            DockerImageName.parse("redis:6.2.3-alpine");

    @Container
    public static GenericContainer redis = new GenericContainer(imageName)
            .withExposedPorts(6379);

    // Tests to follow...
}
```

In this beginning section of the test, we see the most basic form of wiring up with Testcontainers. We apply the `@Testcontainers` annotation to the test class as a whole, letting the library know that it should watch for containers we require during the test execution. The field marked with `@Container` then requests our specific container image `"redis:6.2.3-alpine"` to start, using the standard Redis port, 6379.

When this test class executes, as shown in figure 14.1, Testcontainers starts up the container we've asked for. Testcontainers will wait for a default timeout (60 seconds) for the first mapped port to be available, so we can be confident the container is ready to talk to. The `redis` field then allows us to get information like the hostname and ports for use later in our test.

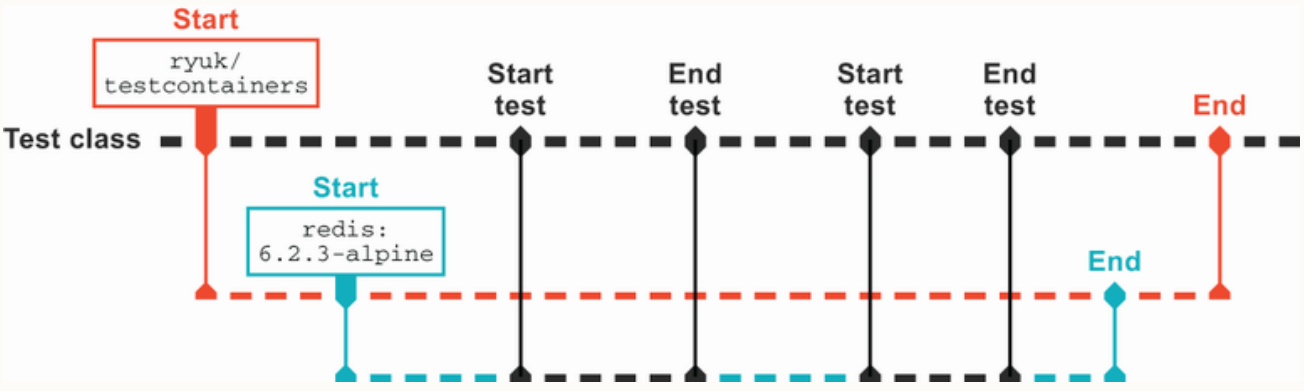


Figure 14.1 Testcontainers execution

With our containerized Redis running, we can get down to the actual tests. Because the key point is our interaction between Redis and the lookup—not how the underlying price

lookup is actually implemented—we can reuse our prior `StubPrice`, which always returns 10 to simplify the testing, as shown here:

```
@Test
  public void cached() {
    var jedis = getJedisConnection();
    jedis.set("price", "20");                          ❶

    CachedPrice price =
      new CachedPrice(new StubPrice(), jedis);         ❷
    BigDecimal result = price.getInitialPrice();

    assertEquals(new BigDecimal("20"), result);        ❸
  }

  @Test
  public void noCache() {
    var jedis = getJedisConnection();
    jedis.del("price");                                ❹

    CachedPrice price = new CachedPrice(new StubPrice(), jedis);
    BigDecimal result = price.getInitialPrice();

    assertEquals(new BigDecimal("10"), result);
  }

  private Jedis getJedisConnection() {                 ❺
    HostAndPort hostAndPort = new HostAndPort(
                      redis.getHost(),
                      redis.getFirstMappedPort());
    return new Jedis(hostAndPort);
  }
```

❶ Sets a price that differs from our stubbed price in Redis

❷ Passes StubPrice as our lookup, which will return 10, not 20

❸ Asserts that we received the cached value

❹ Removes any previously cached value in Redis with the del call

❺ Helper method for setting up our Jedis instance.

It's important to note how the `getJedisConnection` method uses the configuration from Testcontainers to connect to Redis. Although you may observe that `redis .getHost()` is a common value, such as `localhost`, this isn't necessarily guaranteed in every environment. It's better to ask Testcontainers for such values and protect ourselves from unexpected changes to those values in the future.

Although the automated spinup of containers here is quite convenient, it's worth understanding how to control it more directly. This is especially true if your containers require time to start up, as we'll see with later examples like relational databases with required schemas.

The `@Container` annotation recognizes when it's being applied on a static field versus an instance field, as shown in figure 14.2. When applied to a static field, the container will be spun up once for the duration of the test class's execution. If you instead left the field at an instance level, then each individual test will start and stop the container instead.

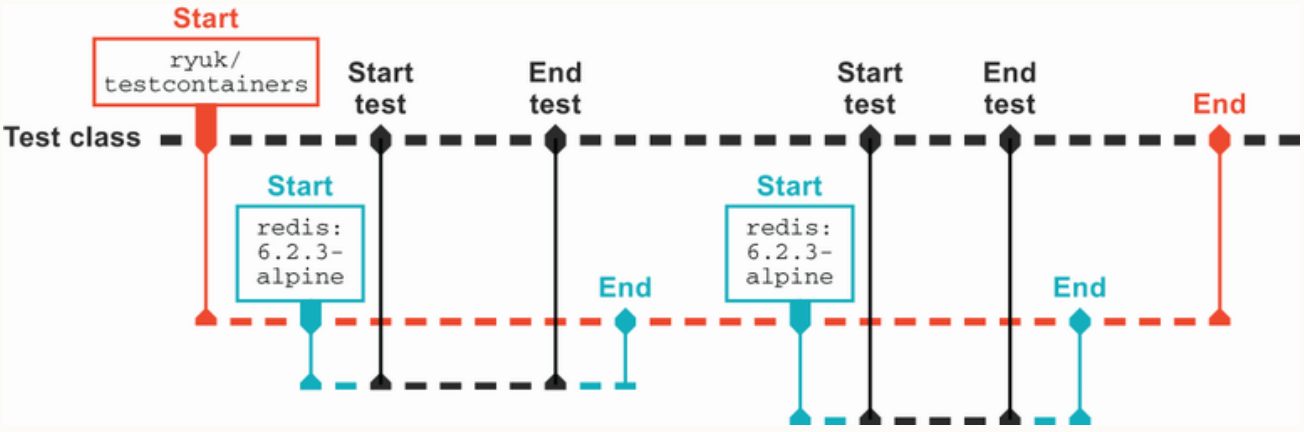


Figure 14.2 Fields and `@Container`

This points to another potential way to manage our container life time: what if we wanted to run the container only once for our entire test suite? To accomplish this, we have to leave the `@Container` annotation behind and use the API directly exposed by the `GenericContainer` object itself as follows:

```
private static final DockerImageName imageName =
    DockerImageName.parse("redis:6.2.3-alpine");

public static GenericContainer redis = new GenericContainer(imageName)
    .withExposedPorts(6379);

@BeforeAll
public void setUp() {
```

```
    redis.start();        ❶
  }
```

❶ start may be safely called multiple times on an instance—it will begin the container only once for each object.

We aren't required to provide a `tearDown` to explicitly stop the container, because the `testcontainers` library takes care of that automatically for us.

Although the previous example calls `start` for each test, the `redis` object could move to a location where it could be shared between multiple test classes safely.

4.1.3 Gathering container logs

If you run these tests at the command line or in your IDE, you may notice that by default there is no output from the containers. For our simple Redis case, this isn't a problem, but more complex setups or debugging may have you wishing for more visibility into those containers. To assist here, Testcontainers allows accessing `STDOUT` and `STDERR` from the containers it spins up.

This support is based on the JDK's `Consumer<>` interface, and several implementations ship with the library. You can connect to standard logging providers or, as we'll demonstrate, get at the raw logging directly.

You may find it inconvenient to have the container logs spewed into your main output, but it's also a pain having to do something custom when you do want them. One solution is to plumb in support to always capture them to a separate location, such as a file in your build output, like this:

```
  @Container                                    ❶
  public static GenericContainer redis =
    new GenericContainer(imageName)
      .withExposedPorts(6379);

  public static ToStringConsumer consumer =              ❷
                      new ToStringConsumer();

  @BeforeAll
  public static void setUp() {
    redis.followOutput(consumer,
                OutputType.STDOUT,
                OutputType.STDERR);             ❸
```

```
    }

    @AfterAll
    public static void tearDown() throws IOException {
        Path log = Path.of("./build/tc.log");              ❹
        byte[] bytes = consumer.toUtf8String().getBytes();
        Files.write(log, bytes,
                StandardOpenOption.CREATE);                ❺
    }
```

❶ We're using the @Container annotation for spinup again because it's so easy.

❷ Our consumer instance will gather the logs during the course of our test run.

❸ Attaches the consumer to our container, asking for both STDOUT and STDERR

❹ Writes to convenient location

❺ Uses java.nio.Files for easy writing of the file contents

4.1.4 An example with Postgres

Redis makes an easy example given its lack of dependencies, the temporary nature of data normally stored there, and the fast startup time on the container. But what about that sticking point in traditional integration testing: the relational database? Often the data we put in a relational store is the most critical to our application's true functionality, but testing it is fraught with stale data, awkward mocking, and false positives.

Testcontainers supports a wide variety of different datastores. These are packaged in separate modules, which must be pulled in. We'll demonstrate using Postgres, but on the Testcontainers website (**https://www.testcontainers.org/modules/databases/**), you'll find a long list of other options.

We include the Postgres module as a test dependency and the main Postgres driver as well to be able to connect to our new database in Maven:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.2.1</version>
</dependency>
<dependency>
```

```
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <version>1.15.3</version>
    <scope>test</scope>
  </dependency>
```

or Gradle:

```
    implementation("org.postgresql:postgresql:42.2.1")
    testImplementation("org.testcontainers:postgresql:1.15.3")
```

It's important that this version match the base `org.testcontainers:testcontainers` library you're using.

A specific class wraps our access to Postgres container. This has helpers for configuring information such as the database name and credentials, as shown here:

```
public static DockerImageName imageName =
        DockerImageName.parse("postgres:9.6.12"));

    @Container
    public static PostgreSQLContainer postgres =
      new PostgreSQLContainer<>(imageName)
        .withDatabaseName("theater_db")
        .withUsername("theater")
        .withPassword("password");
```

All the same lifecycle management considerations apply here, with the added wrinkle that a relational database needs schema applied before it's usable. Many common database migration projects can be run from code, but we'll demonstrate just using JDBC directly to show that nothing magic is going on.

First off, we need a connection to our container instance. Using the JDBC classes, we set it up with parameters from our `postgres` Testcontainer object like this:

```
    private static Connection getConnection() throws SQLException {
      String url = String.format(
          "jdbc:postgresql://%s:%s/%s",
          postgres.getHost(),
          postgres.getFirstMappedPort(),
```

```
            postgres.getDatabaseName());

    return DriverManager.getConnection(url,
                                       postgres.getUsername(),
                                       postgres.getPassword());
}
```

NOTE Testcontainers includes a feature where you can modify your connection strings, and it will automatically start containers for your databases. Although convenient, it's less direct to demonstrate. This may be especially valuable, though, when integrating Testcontainers into an existing test suite.

With our connection, we want to ensure our schema is in place before any of our tests run. Within the scope of one test class, we'd accomplish this with a `@BeforeAll` as follows:

```
@BeforeAll
public static void setup() throws SQLException, IOException {
    var path = Path.of("src/main/resources/init.sql");
    var sql = Files.readString(path);                    (1)
    try (Connection conn = getConnection()) {
        conn.createStatement().execute(sql);             (2)
    }
}
```

(1) For our example, a SQL file has our schema definitions.

(2) Applies the SQL

With the schema in place, our tests can now run against this full-fledged, empty Postgres database as shown next:

```
@Test
public void emptyDatabase() throws SQLException {
    try (Connection conn = getConnection()) {
        Statement st = conn.createStatement();
        ResultSet result = st.executeQuery("SELECT * FROM prices");
        assertEquals(0, result.getFetchSize());
    }
}
```

If you have other abstractions such as DAO (data access objects), repositories, or other ways of reading from the database, they should all work fine with the connection to the container.

4.1.5 An example for end-to-end testing with Selenium

The move to using external resources in containers is a natural fit for integration testing. Similar techniques apply with end-to-end testing as well. Though it depends on your precise system, often an end-to-end test will want to exercise a browser to ensure that a web application is running as expected.

Historically, driving a web browser from code was a touchy proposition. The techniques remain fragile and slow, but Testcontainers takes away the installation and configuration pain by letting you spin up a browser inside a container and control it there remotely.

As with our Postgres example, we'll need to pull in dependencies. In this case, there's a module for Testcontainers support alongside the libraries required for our tests to remote-control the browser instance in Maven:

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>selenium</artifactId>
  <version>1.15.3</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-remote-driver</artifactId>
  <version>3.141.59</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-chrome-driver</artifactId>
  <version>3.141.59</version>
  <scope>test</scope>
</dependency>
```

or Gradle:

```
  testImplementation("org.testcontainers:selenium:1.15.3")
  testImplementation(
```

```
    "org.seleniumhq.selenium:selenium-remote-driver:3.141.59")
  testImplementation(                                          (1)
    "org.seleniumhq.selenium:selenium-chrome-driver:3.141.59")
```

(1) Support for other web browsers also exists in similarly named packages.

Specific classes configure the browser instances. We'll pass in `ChromeOptions` here to indicate that we're starting that particular browser:

```
@Container
public static BrowserWebDriverContainer<?> chrome =
  new BrowserWebDriverContainer<>()
    .withCapabilities(new ChromeOptions());
```

With this instance, we can now write tests that visit web pages and inspect the results as follows:

```
@Test
public void checkTheSiteOut() {
  var url = "https://github.com/well-grounded-java";
  RemoteWebDriver driver = chrome.getWebDriver();
  driver.get(url);                                    (1)

  WebElement title =
          driver.findElementByTagName("h1");          (2)
  assertEquals("well-grounded-java", title.getText());
}
```

(1) Navigates to the GitHub organization for well-grounded-java

(2) Once the page loads, checks the first <h1> contents

This simple example already shows the sort of fragility that end-to-end testing is prone to. What if GitHub redesigns and decides to add another `<h1>` in the page? What if they alter the title text in some subtle way? If you're testing your own applications, this may be less of a problem, but the tight coupling to the presentation remains an issue.

Running inside a container, if things aren't what we expect, it can be difficult to understand why. Fortunately, we can get visual feedback in a couple of ways.

First off, we can screenshot at specific points in time like this:

```
@Test
public void checkTheSiteOut() {
   RemoteWebDriver driver = chrome.getWebDriver();
   driver.get("https://github.com/well-grounded-java");

   File screen = driver.getScreenshotAs(OutputType.FILE);
}
```

The file returned is temporary and will be removed at the end of the test, but you can copy it elsewhere in the code after it's been created.

Seeing more than just a point in time is common enough. You can also just request a video of the session to be recorded automatically as follows:

```
private static final File tmpDirectory = new File("build");

@Container
public static BrowserWebDriverContainer<?> chrome =
   new BrowserWebDriverContainer<>()
      .withCapabilities(new ChromeOptions())
      .withRecordingMode(RECORD_ALL,
                  tmpDirectory,
                  VncRecordingFormat.MP4);
```

As we did with container logs, this will make recordings in our build output any time the tests are run. Everything we need to debug is ready, right there already, should trouble arise.

This just scratches the surface of what Testcontainers will allow you to accomplish. Now let's take a look at leaving JUnit behind to write our tests in a different, potentially more readable form.

14.2 Specification-style testing with Spek and Kotlin

The way JUnit uses methods, classes, and annotations is very natural to a Java developer. But whether or not we're aware of it, it shapes how we express and group our tests. Although not required, we often end up with one test class mapping to our production class and loose clusters of test methods for each implementation method.

An alternative idea is what's known as writing *specifications.* This grew out of frameworks such as RSpec and Cucumber, and rather than focusing on how our code is shaped, it aims to support specifying how the system works at a higher level, more aligned to how humans would discuss requirements.

An example of this sort of testing is available in Kotlin via the Spek framework (see **https://www.spekframework.org/**). As we'll see, many of Kotlin's built-in features allow for a very different organization and feel in our specs.

Installing Spek follows the typical process for dependencies. Spek focuses entirely on how we structure our specifications and leans on the ecosystem for functionality such as assertions and test running. For simplicity here, we'll demonstrate with the assertions and test runner from JUnit 5, but you are not required to use these if you have other libraries you prefer.

In Maven, the `maven-surefire-plugin` from section 11.2.6 just needs to be informed about our specification files, which we'll mark by including `Spek` in the filenames, as shown next. We'll also need the Kotlin support described in section 11.2.5 (not repeated here for length):

```
<build>
 <plugins>
  <plugin>
   <artifactId>maven-surefire-plugin</artifactId>
   <version>2.22.2</version>
   <configuration>
    <includes>
     <include>**/*Spek*.*</include>          (1)
    </includes>
   </configuration>
  </plugin>
 </plugins>
</build>

<dependencies>
 <dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>     (2)
  <version>5.7.1</version>
  <scope>test</scope>
 </dependency>
 <dependency>
  <groupId>org.spekframework.spek2</groupId>
```

```
    <artifactId>spek-dsl-jvm</artifactId>
    <version>2.0.15</version>
    <scope>test</scope>
   </dependency>
   <dependency>
    <groupId>org.spekframework.spek2</groupId>
    <artifactId>spek-runner-junit5</artifactId>     3
    <version>2.0.15</version>
    <scope>test</scope>
   </dependency>
  </dependencies>
```

1 Because of our custom file convention, we have to tell Maven what to run.

2 Uses JUnit's assertion API

3 Uses Spek's integration with JUnit test runners

In Gradle, we plug into the standard `test` task and notify the JUnit platform of Spek's engine, as shown in the next code snippet. You may find that command-line tests will see our specifications without the engine line, but other systems like our IDE may miss them:

```
dependencies {
 testImplementation(                                    1
    "org.junit.jupiter:junit-jupiter-api:5.7.1")

 testImplementation("org.spekframework.spek2:spek-dsl-jvm:2.0.15")
 testRuntimeOnly(                                       2
    "org.spekframework.spek2:spek-runner-junit5:2.0.15")
}

tasks.named<Test>("test") {                             3
 useJUnitPlatform() {
  includeEngines("spek2")                               4
 }
}
```

1 Uses JUnit's assertion API

2 Uses Spek's integration with JUnit test runners

❸ Looks up the test task, informing that it's of type Test so we can access the useJUnitPlatform and following methods

❹ Notifies JUnit of our engine for better IDE integration

Now we can get down to writing our first specification. To examine this, we'll take the prior testing we've done against the `InMemoryCachedPrice` class and see how Spek alters the structure and flow of our testing as follows:

```
import org.spekframework.spek2.Spek
import org.junit.jupiter.api.Assertions.assertEquals
import java.math.BigDecimal

object InMemoryCachedPriceSpek : Spek({
    group("empty cache") {
        test("gets default value") {
            val stubbedPrice = StubPrice()
            val cachedPrice = InMemoryCachedPrice(stubbedPrice)

            assertEquals(BigDecimal(10), cachedPrice.initialPrice)
        }

        test("gets same value when called again") {
            val stubbedPrice = StubPrice()
            val cachedPrice = InMemoryCachedPrice(stubbedPrice)

            val first = cachedPrice.initialPrice
            val second = cachedPrice.initialPrice
            assertTrue(first === second)            ❶
        }
    }
})
```

❶ === is Kotlin's operator for reference equality, so this checks that we get the exact same object between calls, not just an identical value.

Our first specification spells out behavior around an empty cache. We can see a number of Kotlin features at work. First off, our specification is declared as a singleton `object` instead of a class. This helps to clarify test lifetime issues that occasionally happen in JUnit, depending on whether the test runner constructs a single instance of the test per class or per individual test method.

The main specification is declared within a lambda expression, passed as a parameter to the `Spek` class. In this lambda, two important functions are available: `group` and `test`. Each of these is given a full `String` description. No camel-casing, underscores, or other tricks are required to make the description readable. `group` is intended for you to put together various related `test` calls. The `group` constructs can also be nested, if desired.

If this reformatting was all that specification-style testing brought to the table, it wouldn't be very compelling. However, the grouping is more than just naming because we can declare *fixtures* that share the setup across multiple tests as follows:

```
object InMemoryCachedPriceSpek : Spek({
    group("empty cache") {
        lateinit var stubbedPrice : Price
        lateinit var cachedPrice : InMemoryCachedPrice

        beforeEachTest {
          stubbedPrice = StubPrice()
          cachedPrice = InMemoryCachedPrice(stubbedPrice)
        }

        test("gets default value") {
            assertEquals(BigDecimal(10), cachedPrice.initialPrice)
        }

        test("gets same value when called again") {
            val first = cachedPrice.initialPrice
            val second = cachedPrice.initialPrice
            assertTrue(first === second)
        }
    }
})
```

In our "empty cache" group, we declare two fixtures: a `stubbedPrice` for use in setting up the cache and the `cachedPrice` instance we'll test. Any `test` call that's a member of this `group` gets an identical view of these fixtures.

The recommended pattern for fixtures is to use `lateinit` and initialize them in `beforeEachTest`. This need for late initialization actually reflects that Spek runs our specification in two phases: discovery and then execution.

During the discovery phase, the top-level lambda for our specification is run. `group` lambdas are eagerly evaluated, but `test` calls aren't made yet; instead, they are noticed

for later execution. After all of the specification's groups have been evaluated, the `test` lambdas are executed. This separation, shown next, allows for tighter control over the context of each `group` before each individual `test` runs:

```
object InMemoryCachedPriceSpek : Spek({
  group("empty cache") {                          (1)
    lateinit var stubPrice : Price                (1)
    lateinit var cachedPrice : InMemoryCachedPrice (1)

    beforeEachTest {                              (2)
      stubPrice = StubPrice()                     (2)
      cachedPrice = InMemoryCachedPrice(stubPrice) (2)
    }                                             (2)
                                                  (2)
    test("gets default value") {                  (2)
      assertEquals(BigDecimal(10),                (2)
                   cachedPrice.initialPrice)      (2)
    }                                             (2)
                                                  (2)
    test("gets same value when called again") {   (2)
      val first = cachedPrice.initialPrice        (2)
      val second = cachedPrice.initialPrice       (2)
      assertTrue(first === second)                (2)
    }
  }
})
```

(1) Runs during the discovery phase

(2) Runs during the execution phase

The use of `lateinit` is a little clunky, so Spek wraps that up using Kotlin's *delegated properties*. Each fixture can be followed with a `by memoized` call and a lambda to provide the value.

NOTE *memoized* (not memorized!) is a term for a value that's calculated once and cached for later use.

Don't use these for the result of actions you're testing—those should be done within the `test` lambdas themselves, like this:

```
object InMemoryCachedPriceSpek : Spek({
  val stubbedPrice : Price by memoized { StubPrice() }

  group("empty cache") {
    val cachedPrice by memoized { InMemoryCachedPrice(stubbedPrice) }

    test("gets default value") {
      assertEquals(BigDecimal(10), cachedPrice.initialPrice)
    }

    test("gets same value when called again") {
      val first = cachedPrice.initialPrice
      val second = cachedPrice.initialPrice
      assertTrue(first === second)
    }
  }
})
```

The test discovery phase happening via plain execution of our Kotlin code allows for much simpler parameterization than is available in JUnit. Rather than needing additional annotations and reflection-based lookups, we can just loop and repeat calls to `test` as follows:

```
object InMemoryCachedPriceSpek : Spek({
  group("parameterized example") {
    listOf(1, 2, 3).forEach {
      test("testing $it") {          ❶
        assertNotEquals(it, 0)
      }
    }
  }
})
```

❶ Use of it on each time through loop gives us the tests testing 1, testing 2, and testing 3.

For those who may have encountered specification-style testing in other ecosystems, such as RSpec in Ruby or Jasmine in JavaScript, you can substitute the `group` and `test` methods with `describe` and `it` instead for an even more natural narrative flow, like this:

```
object InMemoryCachedPriceSpek : Spek({
  val stubbedPrice : Price by memoized { StubPrice() }
```

```
    describe("empty cache") {
      val cachedPrice by memoized { InMemoryCachedPrice(stubbedPrice) }

      it("gets default value") {
        assertEquals(BigDecimal(10), cachedPrice.initialPrice)
      }

      it("gets same value when called again") {
        val first = cachedPrice.initialPrice
        val second = cachedPrice.initialPrice
        assertEquals(true, first === second)
      }
    }
})
```

Another common format for writing specifications is *Gherkin syntax* (**https://cucumber.io/docs/gherkin/reference/**), popularized by the Cucumber testing tool. This syntax declares our specification in a series of given-when-then statements: *given* this setup, *when* this action happens, *then* we see these consequences. Enforcing this structure often makes specifications more readable as natural language, not just code.

Restating a prior test in Gherkin style could look like this: *Given* an empty cache, *when* calculating the price, *then* we look up the default value. Here's how that translates to Spek's Gherkin support:

```
object InMemoryCachedPriceSpekGherkin : Spek({
    Feature("caching") {
      val stubbedPrice by memoized { StubPrice() }

      lateinit var cachedPrice : Price
      lateinit var result : BigDecimal

      Scenario("empty cache") {
        Given("an empty cache") {
          cachedPrice = InMemoryCachedPrice(stubbedPrice)
        }

        When("calculating") {
          result = cachedPrice.initialPrice
        }
```

```
        Then("it looks up the default value") {
          assertEquals(BigDecimal(10), result)
        }
      }
    }
  })
```

You'll notice this also brings additional grouping from Cucumber by dividing our specification by `Feature` and `Scenario` before we apply given-when-then organization.

Specifications give us a different way to order our testing code to better communicate to later readers. But they still require that we write all of our cases out by hand. Clojure presents some different possibilities to explore how we choose our testing data.

14.3 Property-based testing with Clojure

Unlike Java and Kotlin, Clojure comes with a testing framework in its standard library, `clojure.test`. Although we won't cover this library in depth, let's get familiar with the basics before visiting other, more exotic parts of Clojure's testing ecosystem.

14.3.1 clojure.test

We'll exercise our tests via the Clojure REPL, much like we did throughout chapter 10. If you skipped that chapter or it's been a while, now's a good time to review the basics of Clojure if any of these tests are hard to follow.

Although it ships directly with Clojure, `clojure.test` isn't automatically bundled with our code. We need to request the library via `require`. Entering the following in our REPL makes all of the functions in `clojure.test` available with the prefix `test` that we declare via `:as`:

```
user=> (require '[clojure.test :as test])
nil
user=> (test/is (= 1 1))
true
```

Alternatively, we can pick specific functions via `:refer` to use without prefix like this:

```
user=> (require '[clojure.test :refer [is]])
nil
```

```
user=> (is (= 1 1))
true
```

The `is` function represents the base of assertions in `clojure.test`. When the assertion passes, we see the function returns `true`. How about when it fails?

```
user=> (is (= 1 2))

FAIL in () (NO_SOURCE_FILE:1)
expected: (= 1 2)
  actual: (not (= 1 2))
false
```

Any predicate can be used with `is`. For example, here's how we can confirm that a function will throw an exception we expect:

```
user=> (defn oops [] (throw (RuntimeException. "Oops")))   ❶
#'user/oops

user=> (is (thrown? RuntimeException (oops)))
#error {                                    ❷
 :cause "Oops"
 :via
 [{:type java.lang.RuntimeException
   :message "Oops"
   :at [user$oops invokeStatic "NO_SOURCE_FILE" 1]}]
   ...                                      ❸
```

❶ A function to always throw a RuntimeException

❷ We receive an #error value, not a FAIL message. This indicates that the assertion passed.

❸ The error also contains a full stack trace, excluded here for space.

With our assertions, we're now ready to start constructing tests. A primary method for this is the `deftest` function, shown next:

```
user=> (require '[clojure.test :refer [deftest]])
nil
```

```
user=> (deftest one-is-one (is (= 1 1)))
#'user/one-is-one
```

Having defined our test, we now need to execute it. We can do this via the `run-tests` function, which will find all defined tests in our current namespace. For the REPL, a default namespace called `user` exists automatically, and that's where our `deftest` put our test, as shown here:

```
user=> (require '[clojure.test :refer [run-tests]])
nil
user=> (run-tests)

Testing user

Ran 1 tests containing 1 assertions.
0 failures, 0 errors.
{:test 1, :pass 1, :fail 0, :error 0, :type :summary}
```

Obviously writing and running tests in a REPL is good for learning but not supportable for any long-term use in a project. Eventually it's worth setting up a test runner, although unlike the Java world where JUnit is the standout leader, a few competing options exist in Clojure. A few to consider follow:

- Leiningen (**https://leiningen.org/**) is a popular Clojure build tool that includes support for testing, much like Maven and Gradle.
- Cognitect Labs `test-runner` (**https://github.com/cognitect-labs/test-runner**) is a simple test runner built purely on Clojure's native dependency support.
- Kaocha (**https://github.com/lambdaisland/kaocha**) is a full-featured test runner with a focus on modular design to the testing process.

Having said that, we'll continue in our REPL and look now at an interesting ability that comes along with Clojure: data specifications.

4.3.2 clojure.spec

Although Clojure's integration with the JVM means you can naturally work with classes and objects, functional programming doesn't couple behavior as tightly to data. It's common to have functions that operate against data structures composed of basic primitives, particularly with maps fulfilling the data-carrying behavior we associate with classes in object-oriented programming.

This makes it attractive to have better facilities to test the shape and contents of built-in data structures. That's provided in the standard library with `clojure.spec`. As with `clojure.test`, we need to require the library to get access to it, as follows:

```
user=> (require '[clojure.spec.alpha :as spec])
nil
```

NOTE Although `clojure.spec` uses the term "specification," this is an entirely different use of the term from specifications we saw with Spek in Kotlin. `clojure.spec` defines specifications for *data* rather than for *behavior*.

With that library available, we can start making statements about different values with the function `valid?`. This executes the predicate function we pass it against the value and gives us a Boolean in return, as shown next:

```
user=> (spec/valid? even? 10)
true
user=> (spec/valid? even? 13)
false
```

The `conform` function provides us with the next level of checking, shown in the next code sample. If the value passes the predicate, we receive back that value. Otherwise, the return is a keyword `:clojure.spec.alpha/invalid`:

```
user=> (spec/conform even? 10)
10
user=> (spec/conform even? 13)
:clojure.spec.alpha/invalid
```

We can combine different checks together using the `and` function. It's possible to do this directly via writing our own predicate functions, but using the version from `clojure.spec`, illustrated in the next code snippet, means the library understands the combination we're creating. We'll see in a moment how that can give us more information:

```
user=> (spec/conform (spec/and int? even?) 10)
10
user=> (spec/conform (spec/and int? even?) 13)
:clojure.spec.alpha/invalid
```

```
user=> (spec/conform (spec/and int? even?) "not int")
:clojure.spec.alpha/invalid
```

After seeing `and`, it may come as no surprise that there's an `or` function. But the plot thickens if we try to use `or` just like we did with `and`, as shown here:

```
user=> (spec/conform (spec/or int? string?) 10)
Unexpected error (AssertionError) macroexpanding spec/or at (REPL:1:15).
Assert failed: spec/or expects k1 p1 k2 p2..., where ks are keywords
(c/and (even? (count key-pred-forms)) (every? keyword? keys))
```

This error message tells us that `or` expects to include keywords between the predicates we pass it. This may seem like a strange requirement for a simple Boolean function. However, the reason becomes clearer when we look closer at the results from `conform` when given `or` conditions here:

```
user=> (spec/conform (spec/or :a-number int? :a-string string?) "hello")
[:a-string "hello"]
user=> (spec/conform (spec/or :a-number int? :a-string string?) 10)
[:a-number 10]
user=> (spec/conform (spec/or :a-number int? :a-string string?) nil)
:clojure.spec.alpha/invalid
```

The library tells us not just that the value matches our specification—it tells us which branch of our `or` condition fulfilled the spec. Our specification is bringing more than simple yes/no validity. We're finding out *why* the value passed at the same time.

Repeating our specifications is getting tedious, and in a real application, such repetition is an obvious code smell. `clojure.spec` allows registering specifications against a namespaced keyword. Then we just call `conform` with the keyword like this:

```
user=> (spec/def :well/even (spec/and int? even?))
:well/even
user=> (spec/conform :well/even 10)
10
user=> (spec/conform :well/even 11)
:clojure.spec.alpha/invalid
```

The Clojure REPL comes with a handy `doc` function, which integrates nicely with our specifications. When handed a registered keyword, we get a neatly formatted version of the spec as follows:

```
user=> (doc :well/even)
 -------------------------
 :well/even
 Spec
   (and int? even?)
```

Although `conform` provides feedback on how a successful match happened, the `:clojure.spec.alpha/invalid` keyword is rather opaque about failure. The `explain` function leans on the deeper knowledge our specs already have to tell us why a given value fails, shown next:

```
user=> (spec/explain :well/even 10)
Success!
nil
user=> (spec/explain :well/even 11)
11 - failed: even? spec: :well/even
nil
user=> (spec/explain :well/even "")
"" - failed: int? spec: :well/even
nil
```

Now that we've defined reusable specifications for values, we can apply them in our unit tests directly like this:

```
(deftest its-even
   (is (spec/valid? :well/even 4)))

(deftest its-not-even
   (is (not (spec/valid? :well/even 5))))
```

To this point our specifications have focused on checking individual values. When we work with maps, though, there's an additional question: does the shape of the provided data match our expectations? We validate this with the `keys` function.

Let's imagine part of our theater ticketing system is being written in Clojure. We want to confirm any ticket we're passed has an `id` and `amount`. Optionally, we allow `notes`. We

can define a specification for this like so:

```
user=> (spec/def :well/ticket (spec/keys
                        :req [:ticket/id :ticket/amount]
                        :opt [:ticket/notes]))
:well/ticket
```

Note that the keys here are all namespaced with `:ticket`. This is considered good form for Clojure map keys, because it allows us to maintain a distinction between, say, the `amount` a ticket costs and the `amount` of seats available in a given venue. Should you need to use non-namespaced keys, the various functions like `req` provide alternate versions by appending `-un`, such as, `req-un`.

Calling `conform` on a map will validate the presence of the keys we've spelled out. It also allows unspecified keys alongside the required keys, as illustrated next:

```
user=> (spec/conform :well/ticket
                {:ticket/id 1
                :ticket/amount 100
                :ticket/notes "Noted"})
#:ticket{:id 1, :amount 100, :notes "Noted"}

user=> (spec/conform :well/ticket
                {:ticket/id 1
                :ticket/amount 100
                :ticket/other-stuff true})
#:ticket{:id 1, :amount 100, :other-stuff true}

user=> (spec/conform :well/ticket {:ticket/id 1})
:clojure.spec.alpha/invalid
```

Namespacing keys clearly shows its value, though, in how seamlessly it works with our prior value checking. If a key name has a registered spec, then that value will be validated when we `conform`, as follows:

```
user=> (spec/def :ticket/amount int?)
:ticket/amount

user=> (spec/conform :well/ticket
                {:ticket/id 1 :ticket/amount 100})
#:ticket{:id 1, :amount 100]}
```

```
user=> (spec/conform :well/ticket {:ticket/id 1 :ticket/amount "100"})
:clojure.spec.alpha/invalid
```

`clojure.spec` provides a rich set of abilities for validating our data. But Clojure's focus on how we interact with data doesn't end there.

4.3.3 test.check

When we're writing tests, a lot of our time is spent picking good data to exercise our code. Whether it's building out representative objects or finding the values at the edges of our validation, much energy goes into this search for what to test.

Property-based testing turns this relationship on its head. Instead of constructing examples to execute, we instead define *properties* that should hold true for our functions and then feed in randomized data to ensure those properties are true.

NOTE Much of the recent buzz around property-based testing is credited to the Haskell library, QuickCheck (**https://hackage.haskell.org/package/ QuickCheck**). Other languages have equivalents, such as Hypothesis (**https://hypothesis.readthedocs.io/en/latest/**) in Python. In Clojure, this is provided by the `test.check` library.

This paradigm of testing is a significant change from the traditional unit testing most folks have experienced. In the sort of testing we've seen so far, you expect 100% deterministic results. Any flakiness in running the tests is a sign of a poorly written test and should be eradicated.

Why is property-based testing different—not only allowing but relying on randomized data? For one, although the inputs are randomized, failure doesn't indicate a faulty test—it reveals that our understanding of the system, as expressed by the properties we've defined, is wrong. In effect, property-based tests find edge cases our manually selected data might have missed.

This isn't an argument for entirely abandoning more traditional unit tests, either. It's reasonable to supplement our typical testing with property-based tests, especially in areas where incoming data presents a lot of variety that could trip us up.

Unlike `clojure.test` and `clojure.spec`, `test.check` is a separate package, not in Clojure's standard library. To use it in our REPL, we'll have to tell Clojure about this dependency. The simplest way to do this is to put a file called `deps.edn` in the same directory where

we run `clj`. That file instructs Clojure to download the library from the Maven repository as follows:

```
{
 :deps { org.clojure/test.check {:mvn/version "1.1.0"}}
}
```

You'll need to restart the `clj` REPL after creating the `deps.edn` file. You should see messages the first time you start the REPL indicating it's downloading the necessary JARs.

Property-based testing has two big parts: how you define properties to check about your code, and how you generate the randomized data to test those. Let's start off by configuring generators for our data, which may help inspire us for properties we could check.

`test.check` provides its main support for creating randomized data in the `generators` package. We'll pull in the whole package and alias it to `gen` for a little less typing like this:

```
user=> (require '[clojure.test.check.generators :as gen])
nil
```

Two main functions serve as our entry point into generating random data: `generate` and `sample`. `generate` get a single value, and `sample` gets a set of values instead. Each of these functions requires a generator, of which many are built-in. For instance, here we can simulate flipping a coin by generating randomized Boolean values:

```
user=> (gen/generate gen/boolean)
false

user=> (gen/sample gen/boolean)
(true false true false false false true true false false)

user=> (gen/sample gen/boolean 5)
(true true true true true)
```

The basic generators provided by `test.check` cover much of what you need for primitive types in Clojure. Here's a sampling of their usage. You can see the docs at

http://mng.bz/6XoD for further details and additional, optional parameters that some of these generators take:

```
user=> (gen/sample gen/nat)                                    ❶
(0 1 0 2 3 5 5 7 4 5)

user=> (gen/sample gen/small-integer)                          ❷
(0 -1 1 1 2 4 0 5 -7 -8)

user=> (gen/sample gen/large-integer)                          ❸
(-1 0 -1 -3 3 -1 -8 9 26 -249)

user=> (gen/sample (gen/choose 10 20))                         ❹
(11 20 17 16 11 16 14 19 14 13)

user=> (gen/sample gen/any)                                    ❺
(#{} (true) (-3.0) () (Xs/B 553N -4460N) {} #{-3 W_/R? :? \} () #{} [])

user=> (gen/sample gen/string)                                 ❻
("" "" "" "ØI_" "" "rý" "ƒHODÄ" "fÿí'ß" "ü<Ò29eXÔ" ",ÅÆkO®<")

user=> (gen/sample gen/string-alphanumeric)                    ❼
("" "" "3" "G" "pB9" "e2" "oRt98" "l8" "T61T75k4" "b85O5NXt")

user=> (gen/sample (gen/elements [:a :b :c]))                  ❽
(:b :c :b :a :c :b :a :c :a :b)

user=> (gen/sample (gen/list gen/nat))                         ❾
(() (1) (1) (0 2 1) (0 3) (3 3) (1) (1 6 5 1 2 4 4) (4 7 3 4 7 0) (3 2))
```

❶ Small, natural (non-negative) integers

❷ Small integers, including negatives

❸ Larger integers, including negatives

❹ Choose from the provided integer range

❺ Any Clojure value

❻ Any valid Clojure string

❼ Any string of alphanumeric characters

❽ Choose from a list of elements

❾ Create a list based on the provided generator

These generators can be useful for a type of testing referred to as *fuzzing*. The practice of fuzzing, frequently used in the security field, throws varied, and particularly invalid, data at a system to see where it breaks down. Often the examples we test against aren't imaginative enough, particularly when it comes to input from the outside world. Generators allow us an easy way to strengthen our testing with data we wouldn't have thought up.

Imagine our ticketing application allows open text input for notes but would like to try to extract keywords. If our application is internet facing, we never want that function to throw unexpected exceptions. We could fuzz it like so:

```
user=> (defn validate-input [s]
; imagine implementation here that should never throw
)
#'user/validate-input

user=> (deftest never-throws
       (doall (map (gen/sample gen/string)      ❶
                   validate-input)))

user=> (run-tests)

Testing user

Ran 1 tests containing 0 assertions.
0 failures, 0 errors.
{:test 1, :pass 0, :fail 0, :error 0, :type :summary}
```

❶ doall ensures Clojure doesn't lazily ignore our map because its return value is unused.

Fuzzing can be a useful first step, but there are obviously more interesting properties for our functions than "doesn't crash unexpectedly."

Revisiting our theater ticketing system, the owners are now interested in a new feature where people can bid on tickets. A complex algorithm has been purchased from a machine learning consultancy to maximize the number of people who will purchase in a given set of bid prices. The algorithm guarantees that it won't offer a price outside of the ranges of the bids provided.

We haven't received the code yet, but we want to be prepared to check their claims when it does arrive. Until then, we've provided a stub implementation, shown next, which, given a list of bid prices, will randomly pick one:

```
user=> (defn bid-price [prices] (rand-nth prices))
#'user/bid-price
user=> user=> (bid-price [1 2 3])
1
user=> (bid-price [1 2 3])
3
```

Let's examine how we can use `test.check` to define the properties about our bidding function. In addition to the generators that we pulled in earlier, we'll need to require the functions in both `clojure.test.check` and `clojure.test.check.properties`, as shown here:

```
user=>(require '[clojure.test.check :as tc])
nil

user=>(require '[clojure.test.check.properties :as prop])
nil
```

The first property we'll look to check—and most important to theater owners!—is that we'll never return a bid smaller than what someone has offered:

```
user=>(def bigger-than-minimum
  (prop/for-all [prices (gen/not-empty (gen/list gen/nat))]
    (<= (apply min prices) (bid-price prices))))
#'user/bigger-than-minimum
```

There's a lot going on in this small snippet, so let's break it down. First off, our `def bigger-than-minimum` is giving our property a name for referencing later. It's important to remember that this is only defining the property, not actually checking it yet.

The next line declares `prop/for-all`, which is how we state a property we want to check. It's followed by a list that determines how we'll generate data and what to bind those values to. `[prices (gen/not-empty (gen/list gen/nat))]`. `prices` gets each generated value in turn from the generator statement following it. In this case we're asking for a list of natural (non-negative) integers that is not empty.

The final line finally expresses the actual logic of our property. `(<= (apply min prices) (bid-price prices))` finds the minimum value in our generated list, calls our bid function on that same list, and assures the bid isn't smaller than the minimum.

With that, we can now ask `test.check` to run a set of generated values against the property as follows. The `quick-check` function requires a number of iterations to try and a property to check:

```
user=> (tc/quick-check 100 bigger-than-minimum)
{:result true, :pass? true, :num-tests 100,
 :time-elapsed-ms 13, :seed 1631172881794}
```

Our property passed! The other condition that was requested—that we don't offer a price larger than anyone bid—is an easy extension to make from what we've already written, shown next:

```
user=>(def smaller-than-maximum
  (prop/for-all [prices (gen/not-empty (gen/list gen/nat))]
    (>= (apply max prices) (bid-price prices))))
#'user/smaller-than-maximum

user=>(tc/quick-check 100 smaller-than-maximum)
{:result true, :pass? true, :num-tests 100,
 :time-elapsed-ms 13, :seed 1631173295156}
```

Although it's nice that our properties are passing, let's break them and see what happens then. An easy way to do that is to sneak in a little increase to the bidding function and recheck our property, like so:

```
user=>(defn bid-price [prices] (+ (rand-nth prices) 2))
#'user/bid-price

user=>(tc/quick-check 100 smaller-than-maximum)
{:shrunk {:total-nodes-visited 3, :depth 1, :pass? false, :result false,
:result-data nil, :time-shrinking-ms 1, :smallest [(0)]},
:failed-after-ms 5, :num-tests 1, :seed 1631173486892, :fail [(2)] }
```

Now, this looks different! Our check failed as we hoped that it would, and we've got all the information we need to know about the failing case here. In particular, the `:smallest [(0)]` key indicates the precise failing value seen during the run. We've seen the `:seed`

in our prior results. If we want to run the property again with identical values generated, we can pass that seed into the call like this:

```
user=>(tc/quick-check 100 smaller-than-maximum
       :seed 1631173486892)                                    ❶
{:shrunk {:total-nodes-visited 3, :depth 1, :pass? false, :result false,
:result-data nil, :time-shrinking-ms 1, :smallest [(0)]},
:failed-after-ms 5, :num-tests 1, :seed 1631173486892, :fail [(2)] }
```

❶ Passing in the same seed value as before to get the same failure

A point of interest in the response is the key `:shrunk`. When `test.check` finds a failure, it doesn't just stop and report that. It goes through a process of *shrinking*—creating smaller permutations from the failing generated data to find a minimal case. This is incredibly useful, especially with more complex randomized data. Having the smallest, simplest input that will fail is a huge help for debugging.

`test.check` integrates with the base `clojure.test` library. The `defspec` function both defines a test (as in `deftest`) and a property simultaneously, as shown next:

```
user=> (require '[clojure.test.check.clojure-test :refer [defspec]])
nil

user=> (defspec smaller-than-maximum
  (prop/for-all [prices (gen/not-empty (gen/list gen/nat))]
    (>= (apply max prices) (bid-price prices))))
#'user/smaller-than-maximum

user=> (run-tests)
Testing user
{:result true, :num-tests 100, :seed 1631516389835,
 :time-elapsed-ms 36, :test-var "smaller-than-maximum"}

Ran 1 tests containing 1 assertions.
0 failures, 0 errors.
{:test 1, :pass 1, :fail 0, :error 0, :type :summary}
```

The most difficult aspect of property-based testing often isn't the coding but determining the properties themselves. Although our ticketing example and many basic algorithms, like sorting, lend themselves to obvious properties, many real-world scenarios aren't as clear cut.

Here are some ideas where to look for properties in your systems:

- *Validation and boundaries*—If a function has a condition that you'd validate at runtime, such as the limits of a value, the length of a list, or the contents of a string, this is a ripe location for defining a property.
- *Round-tripping data*—A common operation in many systems is transforming data between various formats. Maybe we receive one type of data on our web request and need to convert it to a different shape before storing it in the database. For these cases, we can define properties showing that a value will round-trip successfully through our conversions and back to its original form without loss.
- *Oracles*—Sometimes we end up writing replacements for existing functionality. This may be for performance, better readability, or any number of other reasons. If we have an alternative path that we consider to be the "right" answer, this can serve as a rich source of properties to compare, even if only during development of the replacements.

4.3.4 clojure.spec and test.check

`test.check` provides a rich set of generators for the primitives in Clojure, but we almost always end up working with richer structures. Writing out accurate generators for those more complex shapes can be tedious and difficult.

Fortunately, `clojure.spec` helps close this gap. `clojure.spec` lets us describe our higher-level data structures generically, and it can automatically turn those into `test.check`-compatible generators that would be messy to define by hand.

To refresh, here are the definitions for our ticketing structure—both the map requirements and the constraints on values:

```
user=> (spec/def :well/ticket (spec/keys
                        :req [:ticket/id :ticket/amount]
                        :opt [:ticket/notes]))
:well/ticket

user=> (spec/def :ticket/amount int?)
:ticket/amount

user=> (spec/def :ticket/id int?)
:ticket/id
```

```
user=> (spec/def :ticket/notes string?)
:ticket/notes
```

The `gen` function in `clojure.spec.alpha` will convert a spec into a generator. We can then pass that generator to the same `test.check` functions methods we've used previously to create randomized data like this:

```
user=> (gen/generate (spec/gen :well/ticket))
#:ticket{:notes "fZBvSkOAWERawpNz", :id -3, :amount 233194633}
```

This random ticket already reveals corners we may not have considered in our spec: Do we really want negative IDs? Should we enforce a range on the amount for our tickets? Looks like we've got more specification and testing to do!

Summary

- Testing isn't one-size-fits-all. Different techniques have different strengths. Test code is an excellent spot to mix and match libraries and languages to enhance those strengths.
- Other languages, like Kotlin and Clojure, can open up styles of testing that are harder to accomplish in Java.
- Integration testing—interacting with datastores and other services—can be finicky and error prone. Testcontainers provides smooth integration for approaching these external dependencies, leveraging the knowledge we have of containers from chapter 12.
- How we write our specification influences how we think about our systems. Spek in Kotlin, and similar specification-style testing frameworks elsewhere, provide an alternative to the code-focused JUnit type of tests. We saw how it can level up the communication in our testing.
- Last, we took an entirely different approach to testing from "write an example and check the result" with property-based testing in Clojure. From generating random data, defining global properties of our system, all the way to shrinking failures to the smallest possible input, property-based testing opens new avenues to ensuring the quality of our systems.

Previous chapter

< 13 Testing fundamentals

Next chapter

Part 5. Java frontiers >